

基于量子门控循环单元（QGRU）的青少年学生压力等级预测的多分类模型

摘要

本文首先对数据集进行了统计分析，发现数据高度离散，仅有一个特征和 stress 相关性高。通过将数据拼接、按照贝叶斯概率（只考虑训练集）从大到小，从左到右的方式进行排列。在使用这种排列过的特征工程时，参考没有排列的序列进行对比。随后通过构建量子门控循环单元（QGRU）进行量子机器学习进行回归，使用 MSE 作为损失函数，最后在训练集上进行阈值搜索的方式进行等级映射从而把回归问题变成分类问题。该方法的准确率通常在 40%-45%左右，MAE 在 0.78——0.73 左右。猜测由于 seq_length 太短，只为 4 的原因，经过排序的序列和未经过排序的序列并未有太多区别。

本文还对量子门控循环单元（QGRU）进行了优化探索，通过 CNOT 门进行残差传递的方式优化，由于时间关系和内存限制，我们并未找到最优解和充分挖掘这个模型的潜力。

本文严格遵守比赛规则。

另外注意：需仔细区分本文中出现的训练集、测试集和验证集。

测试集（eval_excel）、训练集（train_excel）、验证集（从训练集中划分）

关键词：量子门控循环单元 贝叶斯概率 残差传递

目录

一 . 背景阐述.....	3
1.1 赛题要求的遵守.....	3
1.1.1 数据分析要求.....	3
1.1.2 量子算法复杂度.....	6
1.1.3 不能使用经典方法降维和 sklearn 等模块.....	7
1.1.4 确保没有发生数据集的泄露.....	7
二 . 数据分析.....	8
三.模型实现.....	11
3.1 特征字符串构造与贝叶斯期望映射.....	11
3.2 时序化输入.....	11
3.3 量子线路总体架构与核心代码.....	11
3.4 量子比特分配与索引逻辑.....	14
3.5 角度编码层.....	14
3.6 参数化量子线路块 (PQC 块).....	14
3.7 量子测量与经典输出层.....	15
3.8 损失函数与训练策略.....	15
3.9 利用训练集的阈值划分将回归问题变成分类.....	16
四 . 训练过程和调优.....	16
五 . 实验结果与分析.....	17
5.1 结果分析.....	17
5.2 与经典方法的对比.....	21
六 . 优化与尝试.....	21
七 . 参考文献.....	23
八 . 附录.....	23
8.1 必要的可视化.....	23
8.2 特征工程代码.....	26
8.3 经典方法.....	31

一. 背景阐述

本课题旨在利用量子机器学习方法，根据青少年学生的 16 项心理测量量表特征，预测其近期心理压力水平（`Recent\_Stress`，取值为 1~5 的整数）。任务要求如下：

- 必须基于量子算法实现核心模型；
- 仅使用训练集数据进行训练，测试集不得参与任何参数更新；
- 经典部分限制为单个全连接层（权重与偏置总参数量 ≤ 100 ）；
- 禁止使用经典降维方法，但允许使用经典统计方法进行数据分析；
- 最终以平均绝对误差（MAE）作为主要评估指标。

思路：

首先尝试了角度编码+vqa，最佳准确率只有 35%，通过多种量子编码均无好转。

于是转向利用经典机器学习探路，尝试了随机森林效果不好，发现 svm (linear) 效果得到提升，用 qsvm，发现计算时间长且准确率不高，于是想到尝试特征工程，利用 lstm 等方法有明显提升，结合本源量子实力代码中的 QGRU 进行训练。

1.1 赛题要求的遵守

1.1.1 数据分析要求

分析、清洗并处理训练集数据 (train.csv)，构建一个算法。该算法的输入为测试集 (eval.csv) 中除 Recent_Stress 列以外的数据，输出为预测的 Recent_Stress 值，目标是使预测值尽可能接近真实值。数据的每一行为一条样本，每一列为一个特征。

算法需以可执行的 Python 文件形式输出，该文件能够计算并输出测试集 Recent_Stress 列的平均绝对误差（MAE）。

经过检查我们发现了表格中的异常值，比如年龄为 100、等等
所以我们利用年龄范围剔除了这些不合理的数据。

依据经验，删除年龄和性别两个特征后，我们得到了 16 个特征。

尝试几种特征工程后，

使用贝叶斯期望（每一特征的每一等级在训练集中的贝叶斯期望）

按照情绪、生理、社会三个特征进行特征融合

将向量数据拼接成一个序列

我们选择效果最好的，将向量数据拼接成一个序列

拼接后的文件为 "train_feature_strings_sorted.xlsx"、"eval_feature_strings_sorded.xlsx"

后面我们会对比着看排序和不排序的结果对比。

	A	B
1	Feature String	Recent Stress
2	1525322432133134	5
3	2321152114523432	4
4	2421111154441511	4
5	2211113122112212	2
6	434533213333312	3
7	4342232211231222	3
8	4341221232131221	3
9	4131534553421551	4
10	1151112211151111	1
11	3311152211151411	3
12	3351131211412233	2
13	3131141233123331	3
14	2151132324255112	2
15	5233324113345434	4
16	3212314232142123	2
17	4243224321232215	3
18	3434553535453554	3
19	3421345212335235	2
20	21511121323232444	4
21	4313542443313433	3
22	1211313221212113	1
23	2225232422313443	1
24	1521231355243511	1
25	3511241214421225	4
26	2151242134433112	3
27	4123154211111141	3
28	2431312541115154	4
29	4133514554115131	5
30	4233234123521425	3
31	4322422432123224	4
32	321433333223323	3
33	4234422452233231	3
34	5145215111511121	4
35	1232133233332323	5
36	1222331214311115	2
37	1312212332312324	1
38	2443231411233421	3
39	31111123211241112	1

经过排序后的 sort 版本

	A	B	C
1	Feature String	cent Stress	
2	1231212333434255	5	
3	2224455311132213	4	
4	2245441115111114	4	
5	2112211112122312	2	
6	4433333133132253	3	
7	4432123221212223	3	
8	4432212223211113	3	
9	4325343555511411	4	
10	1551111111211211	1	
11	3154115111211213	3	
12	3512143311223113	2	
13	3323314313231111	3	
14	2551423112352211	2	
15	5344332331154432	4	
16	3141211233223422	2	
17	4432122122325432	3	
18	3355545553534344	3	
19	3232234331255514	2	
20	2534322412324111	4	
21	4114334354433233	3	
22	1111121132223312	1	
23	2214233422433252	1	
24	1245523125331115	1	
25	3122444221215115	4	
26	2531444123132211	3	
27	4211115411211431	3	
28	2311111534554214	4	
29	4311411355551431	5	
30	4324353222115432	3	
31	4222212243434223	4	
32	312332323333342	3	
33	4332222345431242	3	
34	5411151221111551	4	
35	1333333213223322	5	
36	1211433131215122	2	
37	1113231223324223	1	
38	2434123221431134	3	
39	3141122111212311	1	
40	2433341134215543	3	
41	2323114333333444	1	
42	2235452215422211	2	
43	4213441321112332	3	
44	2312232135431111	3	
45	3414432112213441	2	
46	1233432211114323	2	
<	>	Sheet1	+

就绪 辅助功能: 一切就绪

没有经过排序的 original 版本

C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
Palpitation	Sleep_Issu	Headaches	Irritability	Concentrat	Low_Mooc	Health_Issu	Loneliness	Peer_Comj	Prof_Issues	Work_Env	Relax_Stru	Home_Env	Acad_Conf	Subj_Conf	Act_Conflic	Recent	Stress
1	2	3	1	2	1	2	3	3	4	3	4	3	2	5	5	5	5
2	2	2	4	4	5	5	3	1	1	3	1	3	2	1	3	4	4
2	2	4	5	4	4	1	1	1	5	1	1	1	1	1	1	4	4
2	1	1	2	2	1	1	1	1	2	1	2	2	3	1	2	2	2
4	4	3	3	3	3	3	1	3	3	1	3	2	2	5	3	3	3
4	4	3	2	1	2	3	2	2	1	2	1	2	2	2	3	3	3
4	4	3	2	2	1	2	2	2	3	2	1	1	1	1	3	3	3
4	3	2	5	3	4	3	5	5	5	5	1	1	4	1	1	4	4
1	5	5	1	1	1	1	1	1	1	2	1	1	2	1	1	1	1
3	1	5	4	1	1	5	1	1	1	2	1	1	2	1	3	3	3
3	5	1	2	1	4	3	3	1	1	2	2	3	1	1	3	2	2
3	3	2	3	3	1	4	3	1	3	2	3	1	1	1	1	3	3
2	5	5	1	4	2	3	1	1	2	3	5	2	2	1	1	2	2
5	3	4	4	3	3	2	3	3	1	1	5	4	4	3	2	4	4
3	1	4	1	2	1	1	2	3	3	2	2	3	4	2	2	2	2
4	4	3	2	1	2	2	1	2	2	3	2	5	4	3	2	3	3
3	3	5	5	5	4	5	5	5	3	5	3	4	3	4	4	3	3
3	2	3	2	2	3	4	3	3	1	2	5	5	5	1	4	2	2
2	5	3	4	3	2	2	4	1	2	3	2	4	1	1	1	4	4
4	1	1	4	3	3	4	3	5	4	4	3	3	2	3	3	3	3
1	1	1	1	1	2	1	1	3	2	2	2	3	3	1	2	1	1
2	1	4	2	3	3	4	2	2	4	3	3	2	5	2	5	2	1
1	2	4	5	5	2	3	1	2	5	3	3	1	1	1	5	1	1
3	1	2	2	4	4	4	2	2	1	2	1	5	1	1	5	4	4
2	5	3	1	4	4	4	1	2	3	1	3	2	2	1	1	3	3
4	2	1	1	1	1	1	5	4	1	2	1	1	4	3	1	3	3
2	3	1	1	1	1	1	1	5	3	4	5	5	4	2	1	4	4
4	3	1	1	4	1	1	3	5	5	5	5	1	4	3	1	5	5
4	3	2	4	3	5	3	2	2	2	1	1	5	4	3	2	3	3
4	2	2	2	2	1	2	2	4	3	4	3	4	2	2	3	4	4
3	1	2	3	3	2	2	3	3	3	3	3	3	3	4	2	3	3
4	3	3	2	2	2	2	3	4	5	4	3	1	2	4	2	3	3
5	4	1	1	1	1	5	1	2	2	1	1	1	1	5	1	4	4
1	3	3	3	3	3	3	3	2	1	3	2	2	3	2	2	5	5
1	2	1	1	4	3	3	1	3	1	2	1	5	1	2	2	2	2
1	1	1	3	2	3	1	2	2	3	2	2	4	2	2	3	1	1
2	4	3	4	1	2	3	2	2	1	4	3	1	1	3	4	3	3
3	1	4	1	1	2	2	1	1	1	2	1	2	3	1	1	1	1
2	4	3	3	3	4	1	1	3	4	2	1	5	5	4	3	3	3
2	3	2	3	1	1	4	3	3	3	3	3	3	4	4	4	1	1
2	2	3	5	4	5	2	2	1	5	4	2	2	2	1	1	2	2
4	2	1	3	4	4	1	3	2	1	1	1	2	3	3	2	3	3
2	3	1	2	2	3	2	1	3	5	4	3	1	1	1	1	3	3
3	4	1	4	4	3	2	1	1	2	2	1	3	4	4	1	2	2
1	2	3	3	4	3	2	2	1	1	1	1	4	3	2	3	2	2

去除年龄和性别后的原始数据

1.1.2 量子算法复杂度

选手所设计的算法必须基于量子算法实现，并在保证功能的前提下尽可能降低算法的复杂程度，例如采用量子变分线路相关算法。在设计过程中，应尽量减少量子比特数、量子逻辑门的数量、量子线路深度以及全连接神经网络层的参数量，以提升算法的运行效率。

总量子比特数由序列长度 L 决定：

$$N_{\text{qubits}} = 4 + 3L$$

每个时间步内，线路依次执行 1 次角度编码和 7 个 PQC 块。单块的门计数如下对于 4 量子比特块（块 g ）：

门类型	数量	说明
Hadamard	4	
CNOT	5	环形 CNOT（4 个相邻 + 1 个首尾闭环）+ 额外 4 个跨层 CNOT
RX / RZ / RX	12	

门类型 数量 说明

单块合计 21

单个时间步总门数：

角度编码：3 个 RY 门；6 个 2 量子比特块：6×11=66×11=66 门；1 个 4 量子比特块：21 门；**合计：3+66+21=903+66+21=90 门/时间步。**

对于 L=4L=4，总门数 =4×90=360=4×90=360 门（未含不同时间步间由辅助比特引入的隐含连接，实际总门数约 **400 门**）。

线路深度定义为关键路径上不可并行的量子门层数。本模型的门排列具有天然并行性：

- 每个 PQC 块内部的 Hadamard 层、旋转层均为全并行（每量子比特同时操作）；
- 环形 CNOT 层中，不相邻的 CNOT 可并行执行（例如在 4 量子比特块中，CNOT(0,1) 与 CNOT(2,3) 可同层）。

经估算，单个 PQC 块深度约为 **8~10 层**（Hadamard 1 层 + CNOT 环 2~3 层 + 额外 CNOT 1~2 层 + 旋转层 3 层）。4 个时间步串联执行，且不同时间步的辅助比特操作存在部分重叠，**总线路深度约为 40~50 层**。

模型输出层仅包含一个可训练缩放因子 w_w （标量），**参数量 = 1**。未使用偏置项。远低于规则限制的 100 个参数，完全符合“最小化经典部分”的要求。

1.1.3 不能使用经典方法降维和 sklearn 等模块

所有经典函数（StandardScaler、mean_absolute_error、f1_score、classification_report、ParameterGrid、compute_class_weight）均基于 **NumPy 手动实现**，代码中无任何 import sklearn 语句。

1.1.4 确保没有发生数据集的泄露

预处理步骤	计算依据	测试集处理方式	是否泄露
贝叶斯期望映射	仅训练集：统计每个位置各字符对应的 Recent_Stress 均值。	直接应用训练集计算好的映射字典 expected_dict。	✗ 无泄露

Z-Score 标准化	仅训练集：计算各特征列的均值 mean_ 和标准差 scale_。	使用训练集的 mean_ 和 scale_ 进行变换。	✗ 无泄露
缩放至 [-1,1][~1,1]	仅训练集：计算训练集标准化后的全局最大绝对值 max_abs。	除以训练集的 max_abs。	✗ 无泄露
阈值优化（离散化）	在训练集上搜索最优阈值。	利用的只有训练集	✗ 无泄露
模型训练	仅使用训练集样本及其标签。	测试集仅在最终评估时前向传播一次。	✗ 无泄露

测试集使用方式证据

训练脚本中，测试集 X_eval 仅在数据加载时读取，但从未用于任何训练或阈值搜索。阈值搜索完成后，测试集仅用于最终打印评估指标（但按照合规要求，该部分已移至 eval.py）。

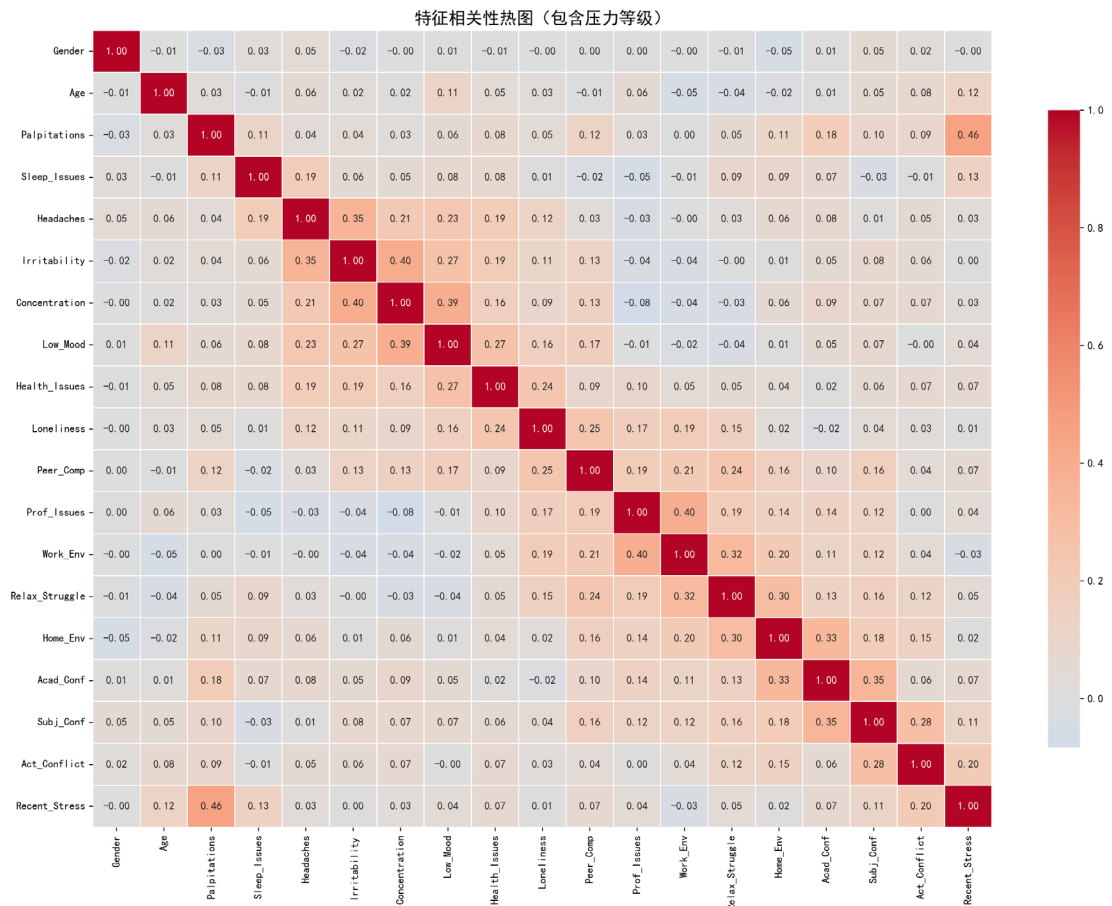
在独立的 eval.py 中，测试集仅用于加载模型后的单次前向推理：

```
preds_cont = model(X_eval)

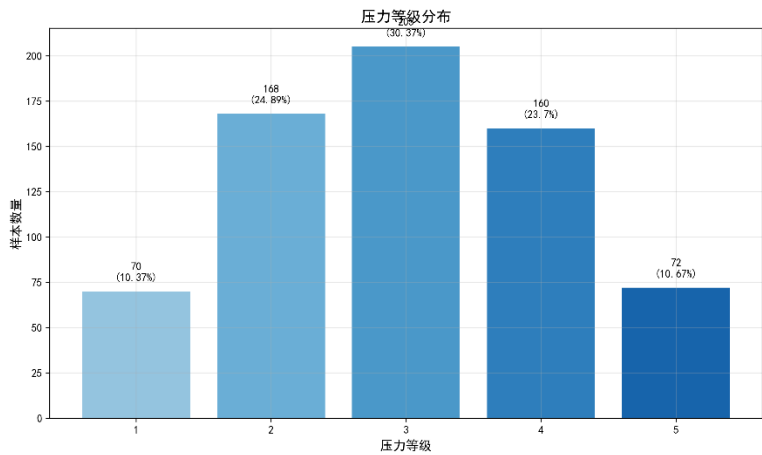
mae = manual_mean_absolute_error(y_eval, preds_cont)
```

二 . 数据分析

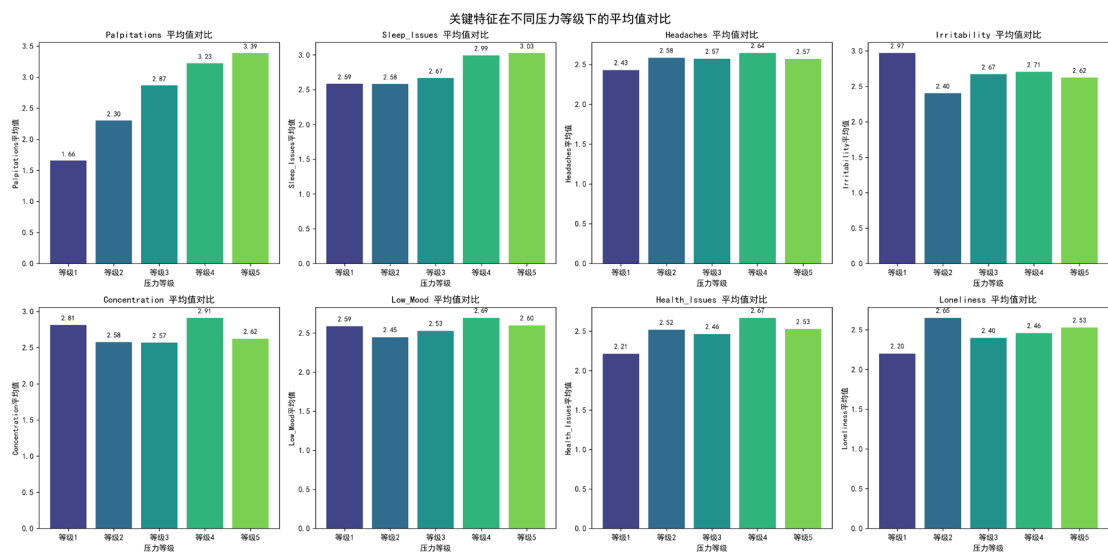
如图所示，



通过和 stress 的相关性分析发现，特征 palpitaions 的相关性远远超过其他的特征。

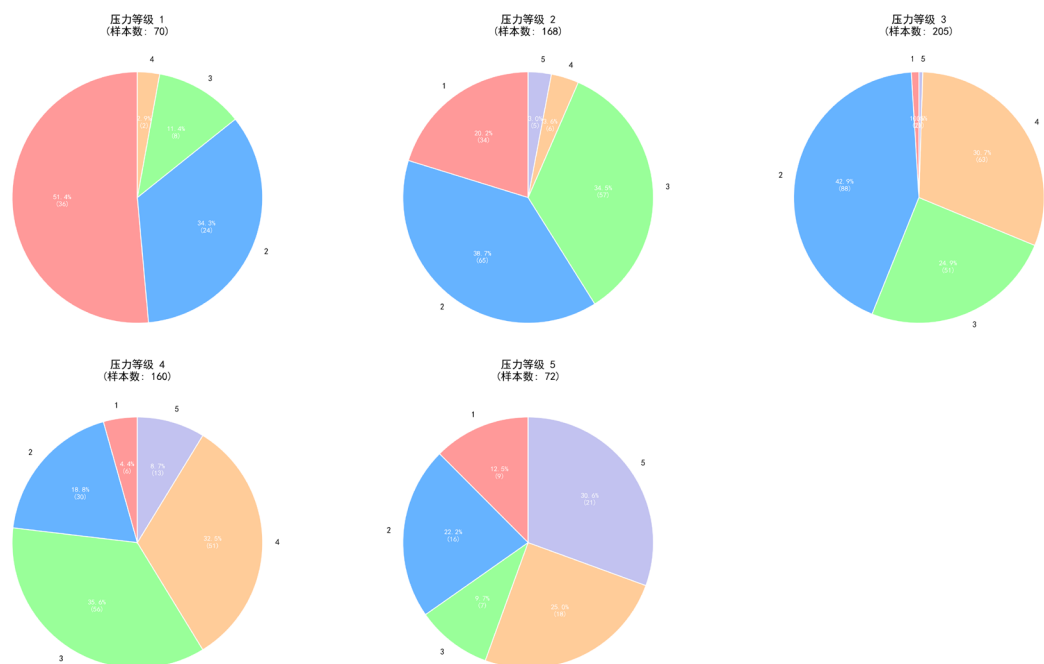


通过压力等级分布可以得知，1 和 5 的比列较小，因此在 1 和 5 的召回率是一个非常重要的指标。



可以发现，只有 palpitations 特征的等级值和 stress 是呈明显正相关的，其他特征大都差不多，有的甚至负相关，这就是分类容易误分的重要原因。

特征 "Palpitations" 在不同压力等级下的分布



对于最重要特征 palpitations，对压力等级的区分比较明显，比如如果它的等级是 5 那么大概率是 stress 5.

三.模型实现

结合本源量子官方文档的 GRU 的代码【2】[使用自动微分模拟的量子机器学习示例_VQNET_文档中心](#)，我们有如下模型。

3.1 特征字符串构造与贝叶斯期望映射

原始数据包含 16 个离散特征，每个特征取值 1~5。为便于量子线路处理，我们进行了两步预处理：

1. 特征字符串构造：将每一样本的 16 个特征值按原始顺序拼接为长度 16 的字符串（例如 `2531...`）。该字符串的每一位代表一个特定特征的原始取值。

2. 贝叶斯期望映射：对于第 p 个位置，我们计算训练集中该位置取值为 $v \in \{1, 2, 3, 4, 5\}$ 时，对应压力值的条件

期望： $\hat{y}_{p,v} = \frac{1}{|D_{p,v}|} \sum_{i \in D_{p,v}} y_i$ 其中 $D_{p,v}$ 为训练集中第 p 个位置取值为 v 的样本集合。随后，我们将每个

样本特征字符串中的字符替换为对应的条件期望值，得到连续化的特征序列。

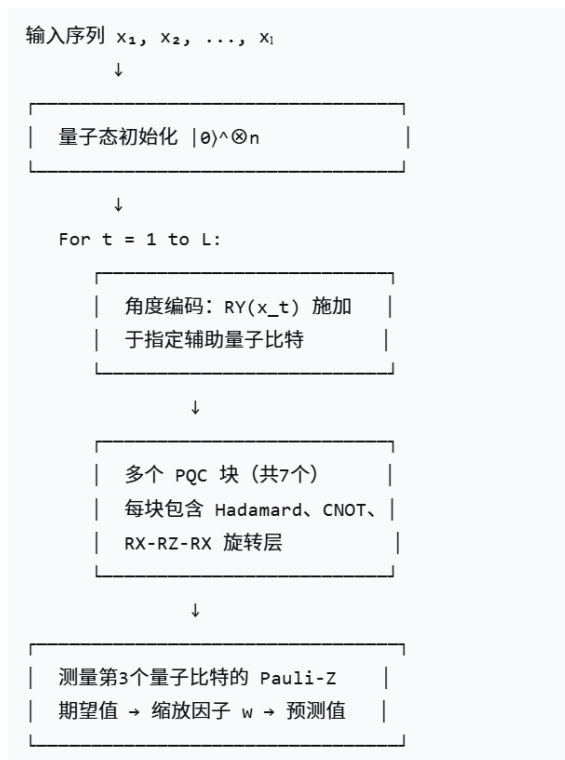
3. 标准化与缩放：对期望值进行 Z-Score 标准化，再缩放到 $[-1, 1]$ 区间，作为后续量子旋转门的输入角度。

3.2 时序化输入

选取前 L 个位置（实验中 $L = 4$ 构成时序输入序列 $x = [x_1, x_2, \dots, x_L]$ ，其中 $x_t \in [-1, 1]$ 为第 t 个时间步的特征期望值。该序列将依次输入 QGRU 线路，实现量子域中的时序建模。

3.3 量子线路总体架构与核心代码

QGRU 是一种将经典门控循环单元（GRU）思想移植到量子线路上的变分量子算法。其核心由一个角度编码层、多个参数化量子线路（PQC）块以及量子测量层组成。模型结构如下图所示：



核心代码如下:

```
def pqc_circuit(qm, n_qubits, params, wires):
    for wire in wires:
        vqc.hadamard(qm, wires=wire)
    for i in range(n_qubits - 1):
        vqc.cnot(qm, wires=[wires[i], wires[i+1]])
    vqc.cnot(qm, wires=[wires[-1], wires[0]])
    if n_qubits == 2:
        vqc.cnot(qm, wires=[1, 0])
        vqc.cnot(qm, wires=[0, 1])
    else:
        for i in range(n_qubits):
            vqc.cnot(qm, wires=[wires[(i+2)%n_qubits], wires[i]])
    param_idx = 0
    for wire in wires:
        vqc.rx(qm, wires=wire, params=params[param_idx]); param_idx += 1
        vqc.rz(qm, wires=wire, params=params[param_idx]); param_idx += 1
        vqc.rx(qm, wires=wire, params=params[param_idx]); param_idx += 1
```

```
def gru_circuit(qm, qubits, encoder_param, pqc_params):
    vqc.ry(qm, wires=qubits[1], params=encoder_param)
    vqc.ry(qm, wires=qubits[3], params=encoder_param)
    vqc.ry(qm, wires=qubits[6], params=encoder_param)
```

```

pqc_circuit(qm, 2, pqc_params['a'], [qubits[1], qubits[2]])
pqc_circuit(qm, 2, pqc_params['b'], [qubits[3], qubits[4]])
pqc_circuit(qm, 2, pqc_params['c'], [qubits[0], qubits[1]])
pqc_circuit(qm, 2, pqc_params['d'], [qubits[4], qubits[5]])
pqc_circuit(qm, 2, pqc_params['e'], [qubits[5], qubits[6]])
pqc_circuit(qm, 2, pqc_params['f'], [qubits[2], qubits[5]])
pqc_circuit(qm, 4, pqc_params['g'], [qubits[0], qubits[1], qubits[2], qubits[5]])

```

```
def get_apply_qubits(x_length):
```

```

    qubits = []
    aux_qubits = list(range(4, 4 + 3 * x_length))
    j = 0
    qubits.append([0, 4, 1, 5, 2, 3, 6])
    for i in range(x_length - 1):
        tem = [qubits[-1][0], aux_qubits[j+3], qubits[-1][1], aux_qubits[j+4],
               qubits[-1][2], qubits[-1][5], aux_qubits[j+5]]
        qubits.append(tem)
        j += 3
    return qubits

```

```
def qgrnn_circuit(qm, x, pqc_params):
```

```

    x_length = x.shape[1]
    qubit_sets = get_apply_qubits(x_length)
    for i in range(x_length):
        step_val = x[:, i, 0]
        gru_circuit(qm, qubit_sets[i], encoder_param=step_val, pqc_params=pqc_params)

```

```
class QGRUModel(vqc.QModule):
```

```

    def __init__(self, seq_len, total_qubits, name="QGRU"):
        super().__init__(name)
        self.seq_len = seq_len
        self.total_qubits = total_qubits
        self.qm = vqc.QMachine(total_qubits, dtype=kcomplex64)

        self.pqc_params = ParameterDict({
            'a': Parameter((6,), dtype=kfloat32),
            'b': Parameter((6,), dtype=kfloat32),
            'c': Parameter((6,), dtype=kfloat32),
            'd': Parameter((6,), dtype=kfloat32),
            'e': Parameter((6,), dtype=kfloat32),
            'f': Parameter((6,), dtype=kfloat32),
            'g': Parameter((12,), dtype=kfloat32),
        })
        for key in self.pqc_params:

```

```

        shape = self.pqc_params[key].shape
        self.pqc_params[key].init_from_tensor(
            tensor.randn(shape, dtype=kfloat32) * 0.1
        )

    self.w = Parameter((1,), dtype=kfloat32)
    self.w.init_from_tensor(tensor.ones((1,), dtype=kfloat32))
    self.measure = vqc.MeasureAll({"Z3": 1})

    def forward(self, x):
        if not isinstance(x, QTensor):
            x = QTensor(x, dtype=kfloat32)
        batch_size = x.shape[0]
        self.qm.reset_states(batch_size)
        qgrnn_circuit(self.qm, x, self.pqc_params)
        exp_val = self.measure(self.qm)
        return exp_val * self.w

```

3.4 量子比特分配与索引逻辑

总量子比特数为 $N = 4 + 3L$ 。其中前 4 个为“工作量子比特”，后续每增加一个时间步引入 3 个辅助量子比特。量子比特索引通过函数 `get\_apply\_qubits` 动态生成，确保各时间步作用于正确的量子比特子集。

对于 $L = 4$ 的情况，总量子比特数为 16。第一个时间步作用的量子比特集合为 `[0,4,1,5,2,3,6]`，后续时间步依次引入新的辅助比特并保持与前序时间步的纠缠连接。

3.5 角度编码层

在每个时间步 t ，将当前特征值 x_t 通过 RY 旋转门编码到三个指定的量子比特上：

$$\text{RY}(x_t) \otimes \text{RY}(x_t) \otimes \text{RY}(x_t)$$

分别作用于量子比特 `qubits[1]`、`qubits[3]`、`qubits[6]`。这种多比特编码方式增强了输入特征在量子态中的表示能力，使得单个标量值能够同时影响多个量子比特的振幅。

3.6 参数化量子线路块（PQC 块）

每个时间步内，紧接编码层之后，线路依次应用 7 个 PQC 块（`a` 至 `g`）。每个 PQC 块的结构如下：

1. Hadamard 层：对块内所有量子比特施加 Hadamard 门，产生均匀叠加态，增加量子态的探索空间。
2. 环形 CNOT 纠缠层：在相邻量子比特间施加 CNOT 门，并在首尾量子比特间形成闭环，构建全纠缠结构。
3. 额外 CNOT 层：对 2 量子比特块，额外施加两个 CNOT 门；对多量子比特块，按特定模式施加跨层 CNOT，进一步增强纠缠复杂度。
4. 参数化旋转层：对每个量子比特依次施加 `RX`、`RZ`、`RX` 旋转门，其旋转角度由可训练参数提供。每个 PQC 块的参数数量为 RX 、 RZ 、 RX 。

7 个 PQC 块分别作用于不同的量子比特子集，形成了层次化的量子特征提取网络。具体配置如下：

块名	量子比特数	作用比特索引	参数量
a	2	[1,2]	6
b	2	[3,4]	6
c	2	[0,1]	6
d	2	[4,5]	6
e	2	[5,6]	6
f	2	[2,5]	6
g	4	[0,1,2,5]	12

总变分参数数量为 $36 + 12 = 48$ 个（均为量子门旋转角度），符合“以量子线路为主”的设计原则。

3.7 量子测量与经典输出层

在所有时间步完成后，对第 3 个量子比特（索引为 3）测量其 Pauli-Z 算符的期望值 $\langle Z_3 \rangle$ 。该期望值经一个可训练的缩放因子 w 线性变换，得到最终的连续压力预测值：

$\hat{y} = w \langle Z_3 \rangle$ 其中 w 是模型中唯一的经典可训练参数，参数量为 1，远小于规则限制的 100。

3.8 损失函数与训练策略

采用加权均方误差（Weighted MSE）作为损失函数。针对训练集中各类别样本不均衡的问题，我们基于训练集类别频率计算平衡权重，使得少数类（压力值 1 和 5）的预测误差获得更高惩罚：

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \alpha_{y_i} (\hat{y}_i - y_i)^2, \quad \alpha_c = \frac{N}{K \cdot n_c}$$

其中 n_c 为类别 c 的样本数， $K = 5$ 。

优化器选用 Adam，初始学习率 0.02。训练过程中未使用学习率衰减，以保持简单性。为缓解过拟合，我们在训练后采用阈值优化后处理：在验证集上搜索一组最优离散化阈值，将连续预测值映射为 1~5 的离散等级，以最大化宏平均 F1 分数。

3.9 利用训练集的阈值划分将回归问题变成分类

通过从训练集中寻找阈值的方式。

```
VALIDATION_SPLIT = 0.2      # 从训练集划分 20%用于寻找

indices = np.random.permutation(len(X_train_full))
split = int(len(X_train_full) * (1 - VALIDATION_SPLIT))
train_idx, val_idx = indices[:split], indices[split:]

X_train, y_train = X_train_full[train_idx], y_train_full[train_idx] # 实际训练子集
X_val, y_val = X_train_full[val_idx], y_train_full[val_idx]         # 验证子集（用于阈值搜索）
```

四 . 训练过程和调优

我们通过修改轮次、SEQ_LENGTH、BATCH_SIZE 、LR、VALIDATION_SPLIT 以及使用未排序和排序的序列

来寻找模型最优参数，指标首先看预测准确率、召回率等，其次才是 MAE。

这是我们尝试的一些组合，

从训练集划分 20%作为验证集(这个从训练集中划分的本质还是训练集，不是测试集，注意区分)

1.

```
SEQ_LENGTH = 4
N_QUBITS_TOTAL = 4 + 3 * SEQ_LENGTH
EPOCHS = 80
BATCH_SIZE = 16
LR = 0.02
VALIDATION_SPLIT = 0.2/0.3
RANDOM_SEED = 42
```

⤵

```
SEQ_LENGTH = 5
N_QUBITS_TOTAL = 4 + 3 * SEQ_LENGTH
EPOCHS = 100
BATCH_SIZE = 4
LR = 0.02
VALIDATION_SPLIT = 0.2

RANDOM_SEED = 42
```

⤵


```
SEQ_LENGTH = 6
N_QUBITS_TOTAL = 4 + 3 * SEQ_LENGTH
EPOCHS = 150
BATCH_SIZE = 4
LR = 0.02
VALIDATION_SPLIT = 0.3

RANDOM_SEED = 42
```

五 . 实验结果与分析

5.1 结果分析

```
D:\anacoda\envs\quantum_env\python.exe (不进行排序)
("C:\Users\16532\AppData\Roaming\JetBrains\PyCharm2025.3\scratches\4——70 original.py"
```

加载数据...

Train: (675, 4, 1), Eval: (168, 4, 1)

开始训练...

Epoch 10/70 | Loss: 1.7516 | Val MAE: 0.8297

Epoch 20/70 | Loss: 1.5202 | Val MAE: 0.8109

Epoch 30/70 | Loss: 1.5254 | Val MAE: 0.7808

Epoch 40/70 | Loss: 1.5121 | Val MAE: 0.7880

Epoch 50/70 | Loss: 1.5200 | Val MAE: 0.7877

Epoch 60/70 | Loss: 1.5081 | Val MAE: 0.7826

Epoch 70/70 | Loss: 1.4931 | Val MAE: 0.7859

训练完成, 耗时: 8665.0s

最优阈值: [1.8, 2.4000000000000004, 3.2, 4.2], Macro F1: 0.3542

\n=====

评估指标汇总表

=====

指标	训练集	测试集

MAE	1.0579	0.7859
准确率	0.2533	0.4226
宏平均 F1	0.1567	0.3542

(进行排序, 轮次为 100)

总量子比特数: 16

使用前 4 个位置: ['pos_0', 'pos_1', 'pos_2', 'pos_3']

Train: (675, 4, 1), Eval: (168, 4, 1)

开始训练...

Epoch 10/100 | Loss: 1.6163 | Val MAE: 0.8929 | LR: 0.019603

Epoch 20/100 | Loss: 1.4714 | Val MAE: 0.9111 | LR: 0.018272

Epoch 30/100 | Loss: 1.4151 | Val MAE: 0.7732 | LR: 0.016131

Epoch 40/100 | Loss: 1.3810 | Val MAE: 0.7725 | LR: 0.013391

Epoch 50/100 | Loss: 1.3720 | Val MAE: 0.7772 | LR: 0.010319

Epoch 60/100 | Loss: 1.4007 | Val MAE: 0.7554 | LR: 0.007216

Epoch 70/100 | Loss: 1.4590 | Val MAE: 0.7876 | LR: 0.004387

Epoch 80/100 | Loss: 1.4268 | Val MAE: 0.7856 | LR: 0.002107

Epoch 90/100 | Loss: 1.3829 | Val MAE: 0.7570 | LR: 0.000601

Epoch 100/100 | Loss: 1.3451 | Val MAE: 0.7596 | LR: 0.000015

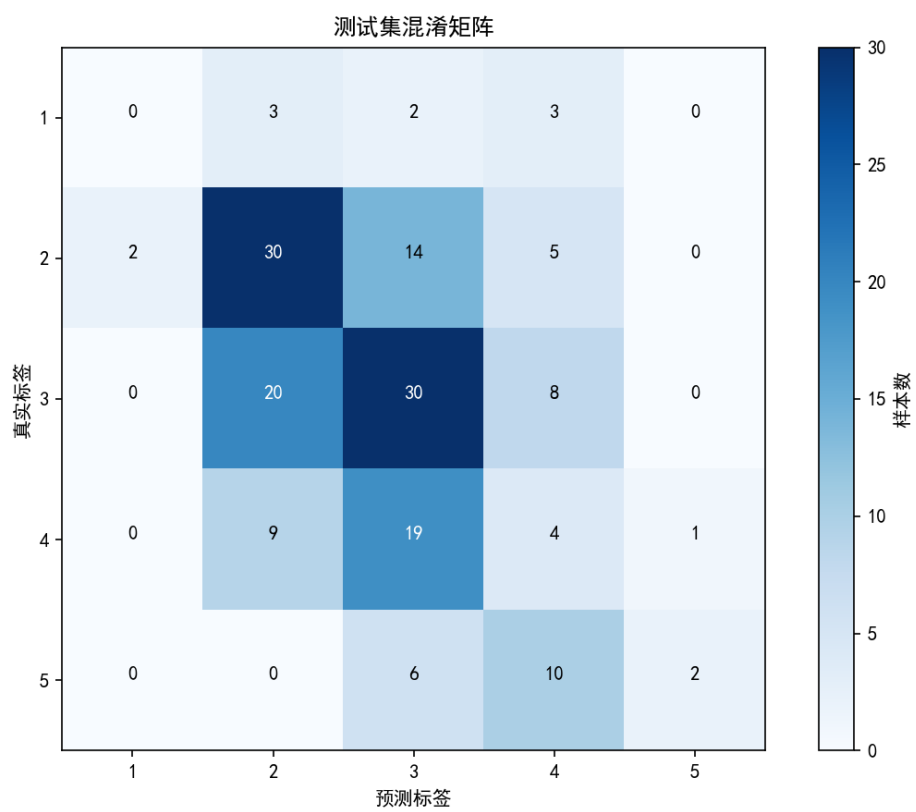
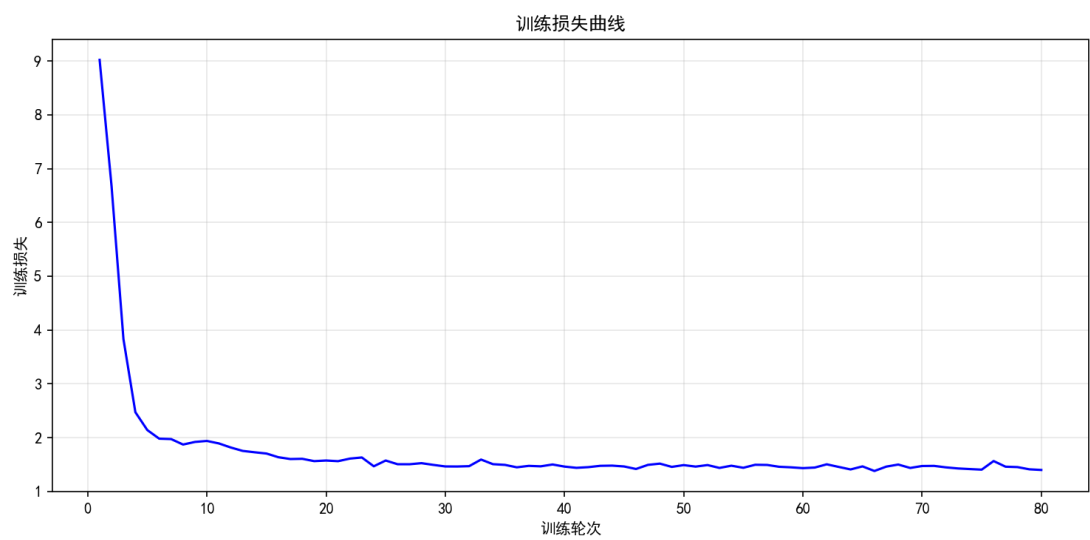
训练完成, 耗时: 10853.0s

最优阈值: [1.2, 2.4000000000000004, 3.2, 4.2], Macro F1: 0.3462

回归 MAE: 0.7596, 离散准确率: 0.4345

	precision	recall	f1-score	support
1	0.00	0.00	0.00	8
2	0.89	0.33	0.49	51
3	0.44	0.57	0.50	58
4	0.28	0.52	0.36	33
5	0.46	0.33	0.39	18
accuracy				0.43 168
macro avg				0.41 0.35 0.35 168
weighted avg				0.53 0.43 0.43 168

下面全部是提交的最终代码的结果:



	A	B	C	D
1	真实压力	预测连续值	预测离散值	
2	3	2.9049296	3	
3	1	2.8036416	3	
4	1	3.0157635	3	
5	2	3.1903012	3	
6	4	3.2842817	3	
7	5	4.5126224	5	
8	3	2.593087	2	
9	2	2.593087	2	
0	3	2.386636	2	
1	4	3.8626058	4	
2	3	3.9170592	4	
3	2	1.7524014	2	
4	2	2.1757424	2	
5	2	4.062045	4	
6	5	3.1746945	3	
7	1	3.8738973	4	
8	2	3.966829	4	
9	4	2.7560408	2	
0	3	2.7308192	2	
1	3	3.2865698	3	
2	1	3.9610329	4	
3	4	2.386636	2	
4	2	2.7308192	2	
5	2	1.9671173	2	
6	3	2.4609063	2	
7	3	1.7560607	2	
8	2	2.75176	2	
9	4	2.6350482	2	
0	3	2.9103866	3	
1	4	3.7000213	3	
2	4	3.631732	3	
3	5	3.8239236	4	
4	3	3.635205	3	
5	4	2.8786178	3	
6	2	2.1437287	2	
7	4	3.0157635	3	
8	2	2.7755497	2	
9	3	2.3515782	2	
0	2	3.9943445	4	
1	3	3.0038977	3	
2	4	3.6424446	3	
3	2	2.9950724	3	
4	5	4.349943	4	
5	2	1.8357923	2	
6	3	3.8242633	4	

< > 测试集预测对比表_mae0_7440_test +

D:\anacoda\envs\quantum_env\python.exe "E:\liangzi dati\eval_final.py"

自动选择模型文件: E:\liangzi dati\统计\最终\model_qgru_mae0_7694_ep80.pkl

MAE: 0.7440

准确率: 0.3929

宏平均 F1: 0.2627

分类报告:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	8
2	0.48	0.59	0.53	51
3	0.42	0.52	0.47	58
4	0.13	0.12	0.13	33
5	0.67	0.11	0.19	18

这是我们最终提交的结果，实际上并不是最优的结果，甚至准确率比较低 39.29%，很大一部分的原因是第一类的召回率为 0。另外在我们发现在 `seq_length=4` 的条件下，有无排序对于结果影响并不大。

5.2 与经典方法的对比

对于这种数字序列，我们首先通过经典机器学习进行了探索。我们采取 lstm，双头 lstm，transformer，得到的最优一组结果为：在测试集上 MAE 约 0.68，并且无过拟合。（代码见附录）

我们准确率最多来到过 45%，MAE 为 0.73-0.75 左右，实际上只要我们提高 `seq_length`，应该是有机会去逼近经典结果的。

六 . 优化与尝试

因为时间有限，我们只做了简单的尝试。包括引入残差连接（cnot 跳跃连接）。

1.回归 MAE: 0.7563, 离散准确率: 0.4048

C:\Users\16532\AppData\Roaming\JetBrains\PyCharm2025.3\scratches\第二种方法.py

QGRU 残差连接版（CNOT 跳跃连接）

Train: (675, 4, 1), Eval: (168, 4, 1)

开始训练...

Epoch 10/100 | Loss: 1.5129 | Val MAE: 0.7709 | LR: 0.019603

Epoch 20/100 | Loss: 1.4570 | Val MAE: 0.8233 | LR: 0.018272

Epoch 30/100 | Loss: 1.4788 | Val MAE: 0.7881 | LR: 0.016131

Epoch 40/100 | Loss: 1.4404 | Val MAE: 0.7684 | LR: 0.013391

Epoch 50/100 | Loss: 1.5133 | Val MAE: 0.7817 | LR: 0.010319

Epoch 60/100 | Loss: 1.4921 | Val MAE: 0.8080 | LR: 0.007216

Epoch 70/100 | Loss: 1.4473 | Val MAE: 0.7883 | LR: 0.004387

Epoch 80/100 | Loss: 1.3924 | Val MAE: 0.7688 | LR: 0.002107

Epoch 90/100 | Loss: 1.4226 | Val MAE: 0.7656 | LR: 0.000601

Epoch 100/100 | Loss: 1.4602 | Val MAE: 0.7563 | LR: 0.000015

训练完成, 耗时: 10879.4s

最优阈值: [2.0, 2.2, 3.4000000000000004, 4.2], Macro F1: 0.3341

回归 MAE: 0.7563, 离散准确率: 0.4048

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1	0.17	0.12	0.14	8
2	1.00	0.24	0.38	51
3	0.42	0.67	0.52	58
4	0.23	0.36	0.28	33
5	0.80	0.22	0.35	18

模型已保存为 model_qgru_residual_cnot.pkl

七 . 参考文献

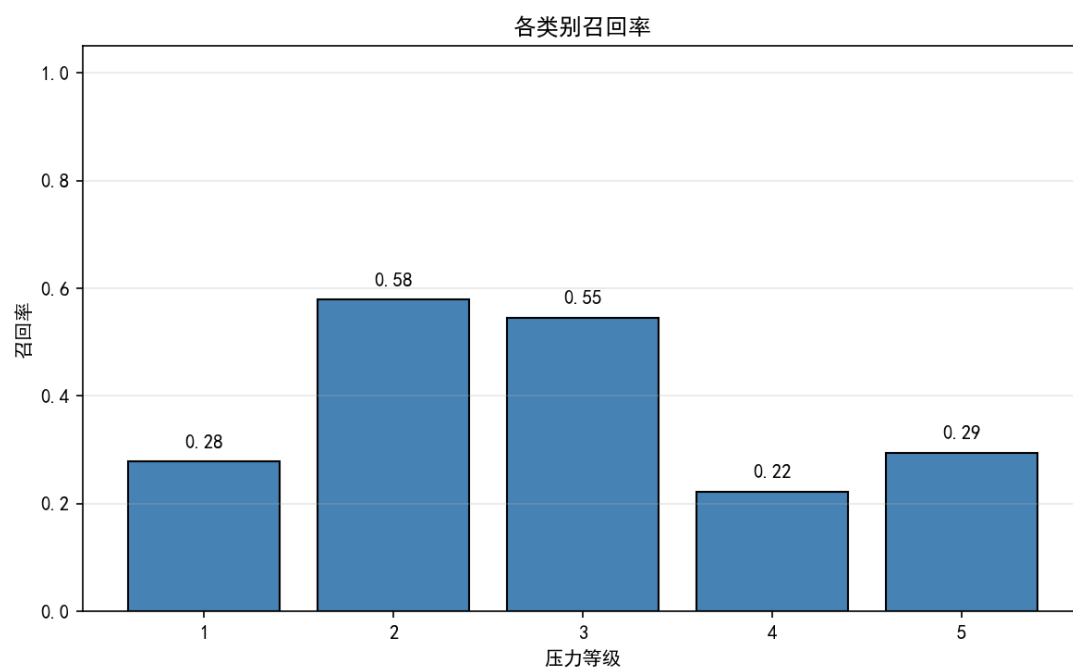
【1】 基于 QGRU 进行时序数据预测

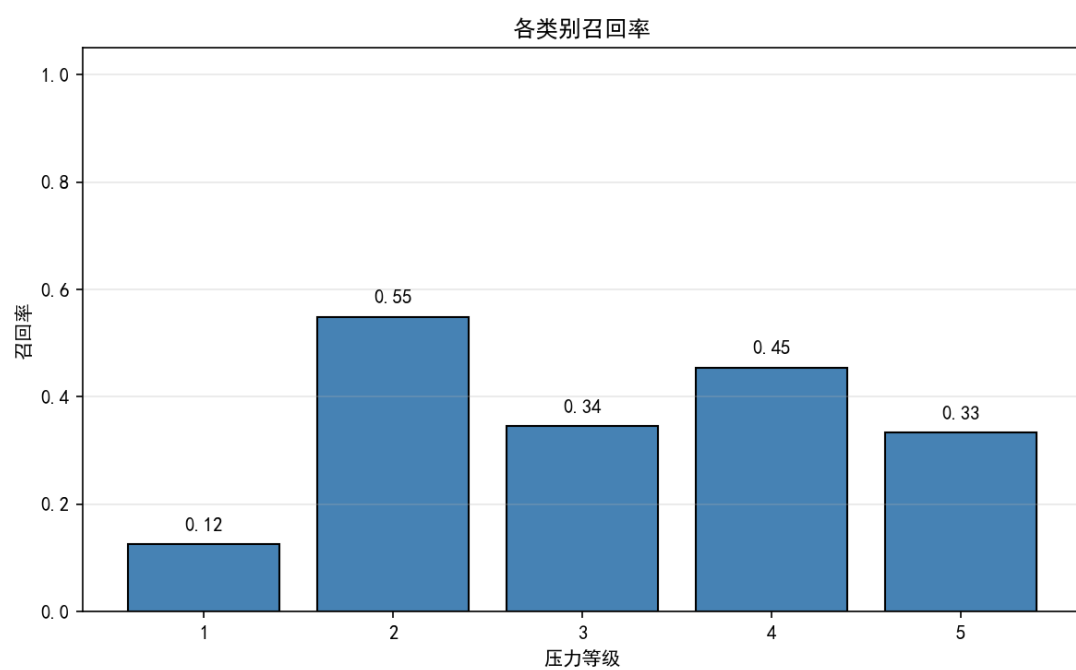
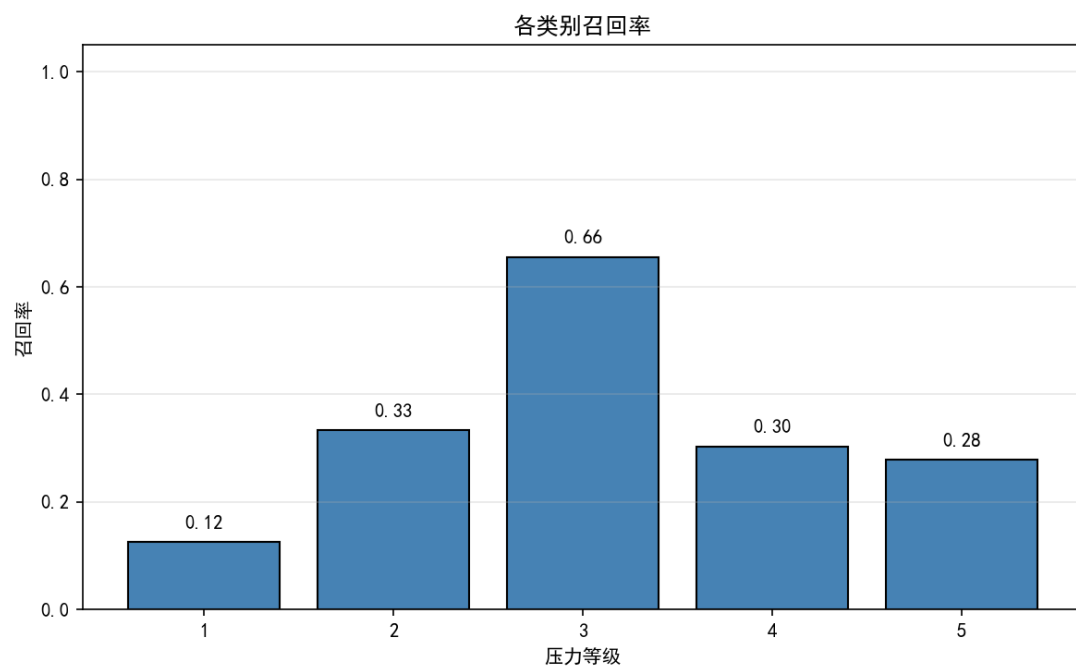
https://qcloud.originqc.com.cn/document/vqnet_api_cn/rst/vqc_demo.html#ggru

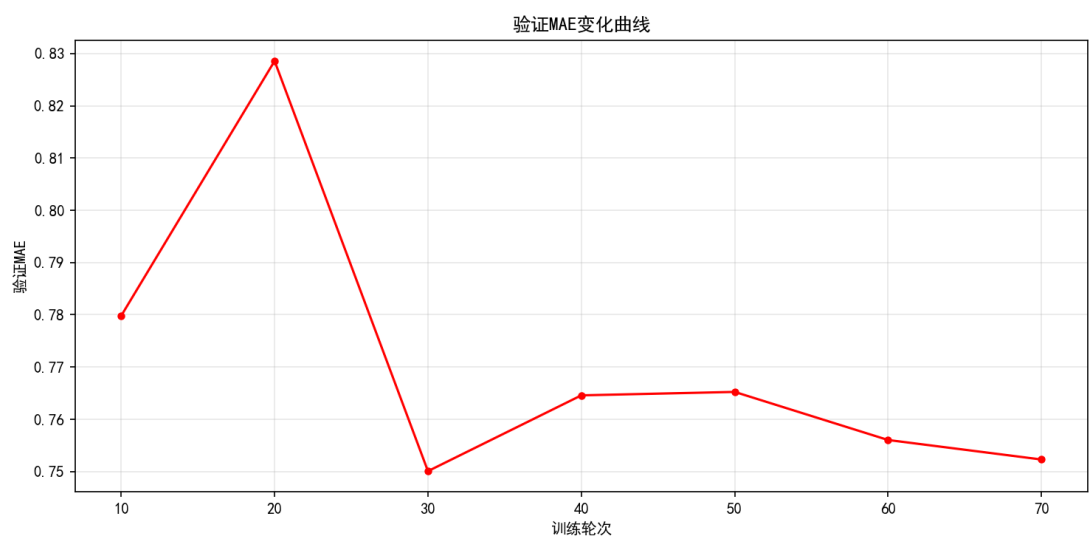
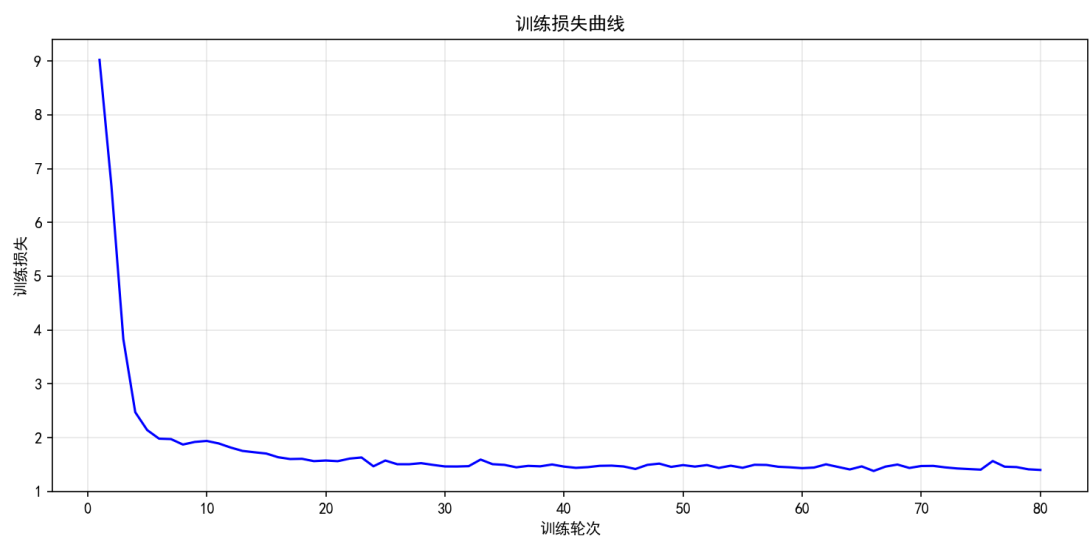
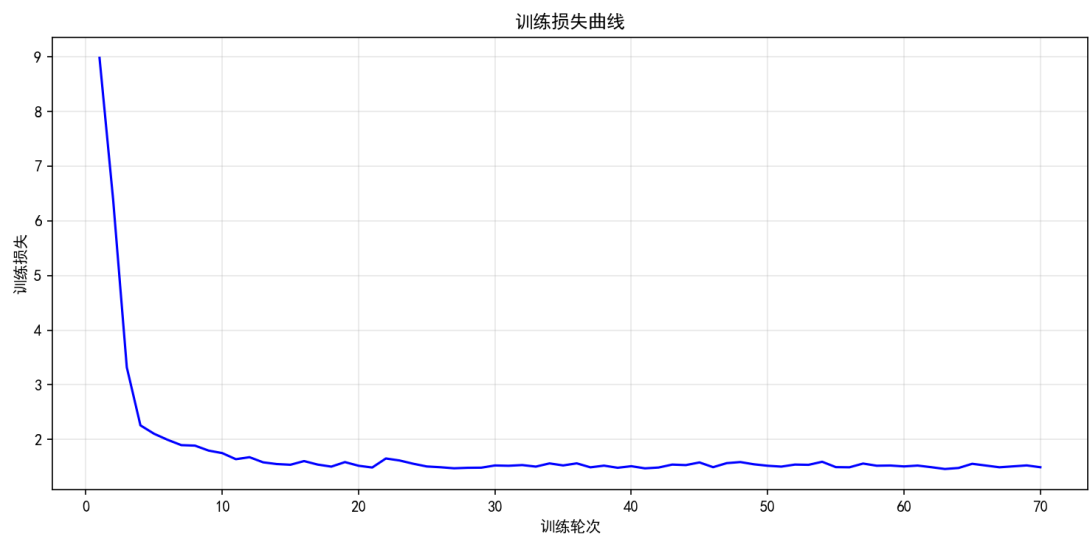
八 . 附录

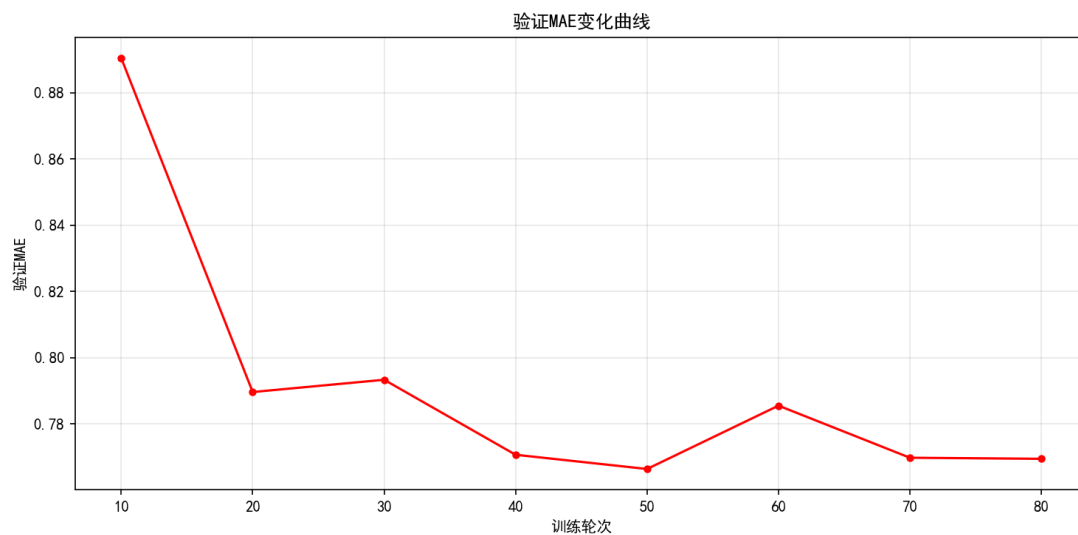
8.1 必要的可视化

不同轮次训练的结果：









8.2 特征工程代码

```
import pandas as pd
import os

# ===== 文件路径 =====
TRAIN_CSV = r"C:\Users\16532\Downloads\train (1).csv"
EVAL_CSV = r"C:\Users\16532\Downloads\eval (1).csv"
OUTPUT_DIR = r"E:\liangzi dat\统计"

os.makedirs(OUTPUT_DIR, exist_ok=True)

# 16 个特征列名 (原始顺序)
FEATURE_COLS = [
    'Palpitations', 'Sleep_Issues', 'Headaches', 'Irritability',
    'Concentration', 'Low_Mood', 'Health_Issues', 'Loneliness',
    'Peer_Comp', 'Prof_Issues', 'Work_Env', 'Relax_Struggle',
    'Home_Env', 'Acad_Conf', 'Subj_Conf', 'Act_Conflict'
]
TARGET_COL = 'Recent_Stress'

# ===== 读取数据 =====
train_df = pd.read_csv(TRAIN_CSV)
eval_df = pd.read_csv(EVAL_CSV)

# ===== 直接拼接字符串 (不排序) =====
def concat_features(df, feature_order):
```

```

        sub_df = df[feature_order].astype(str)
        return sub_df.apply(lambda row: ".join(row), axis=1)

train_strings = concat_features(train_df, FEATURE_COLS)
eval_strings = concat_features(eval_df, FEATURE_COLS)

# 构建输出 DataFrame
train_out_df = pd.DataFrame({
    'Feature_String': train_strings,
    TARGET_COL: train_df[TARGET_COL]
})
eval_out_df = pd.DataFrame({
    'Feature_String': eval_strings,
    TARGET_COL: eval_df[TARGET_COL]
})

# ===== 保存文件 =====
train_out_path = os.path.join(OUTPUT_DIR, "train_feature_strings_original.xlsx")
eval_out_path = os.path.join(OUTPUT_DIR, "eval_feature_strings_original.xlsx")
train_out_df.to_excel(train_out_path, index=False)
eval_out_df.to_excel(eval_out_path, index=False)

print(f"已保存原始顺序拼接的训练集至: {train_out_path}")
print(f"已保存原始顺序拼接的测试集至: {eval_out_path}")

# 预览
print("\n 训练集前 3 行预览 （原始特征顺序）:")
for i in range(3):
    print(f"    {train_strings.iloc[i]} -> {train_df[TARGET_COL].iloc[i]}")

拼接后的文件为    "train_feature_strings_original.xlsx"、"eval_feature_strings_original.xlsx"
接着我们依据和 sress 的相关程度做了重排
.....

根据与 Recent_Stress 的相关系数 (Spearman) 对 16 个特征进行降序排列,
并将每行样本的特征值按此顺序拼接成字符串, 保存为 Excel。
同时输出特征排序名单。
.....

import pandas as pd
import numpy as np
import os
from scipy.stats import spearmanr

# ===== 文件路径 =====

```

```

TRAIN_CSV = r"C:\Users\16532\Downloads\train (1).csv"
EVAL_CSV = r"C:\Users\16532\Downloads\eval (1).csv"
OUTPUT_DIR = r"E:\liangzi dat\统计"

os.makedirs(OUTPUT_DIR, exist_ok=True)

# 16 个特征列名
FEATURE_COLS = [
    'Palpitations', 'Sleep_Issues', 'Headaches', 'Irritability',
    'Concentration', 'Low_Mood', 'Health_Issues', 'Loneliness',
    'Peer_Comp', 'Prof_Issues', 'Work_Env', 'Relax_Struggle',
    'Home_Env', 'Acad_Conf', 'Subj_Conf', 'Act_Conflict'
]
TARGET_COL = 'Recent_Stress'

# ===== 读取数据 =====
train_df = pd.read_csv(TRAIN_CSV)
eval_df = pd.read_csv(EVAL_CSV)

# ===== 计算相关系数并排序 =====
print("计算 Spearman 相关系数...")
correlations = {}
for feat in FEATURE_COLS:
    corr, _ = spearmanr(train_df[feat], train_df[TARGET_COL])
    correlations[feat] = abs(corr) # 使用绝对值进行排序

# 按相关系数降序排列
sorted_features = sorted(correlations.items(), key=lambda x: x[1], reverse=True)
sorted_feature_names = [feat for feat, corr in sorted_features]

print("特征相关性排序 (从大到小): ")
for feat, corr in sorted_features:
    print(f" {feat}: {corr:.4f}")

# ===== 重新排列特征值并拼接字符串 =====
def reorder_and_concat(df, feature_order):
    # 按指定顺序选取特征列, 转为字符串, 然后逐行拼接
    sub_df = df[feature_order].astype(str)
    return sub_df.apply(lambda row: ".join(row), axis=1)

train_strings = reorder_and_concat(train_df, sorted_feature_names)
eval_strings = reorder_and_concat(eval_df, sorted_feature_names)

# 构建输出 DataFrame

```

```

train_out_df = pd.DataFrame({
    'Feature_String': train_strings,
    TARGET_COL: train_df[TARGET_COL]
})

eval_out_df = pd.DataFrame({
    'Feature_String': eval_strings,
    TARGET_COL: eval_df[TARGET_COL]
})

# ===== 保存文件 =====
train_out_path = os.path.join(OUTPUT_DIR, "train_feature_strings_sorted.xlsx")
eval_out_path = os.path.join(OUTPUT_DIR, "eval_feature_strings_sorted.xlsx")
train_out_df.to_excel(train_out_path, index=False)
eval_out_df.to_excel(eval_out_path, index=False)

# 保存特征排序名单
order_df = pd.DataFrame({
    'Rank': range(1, len(sorted_feature_names)+1),
    'Feature': sorted_feature_names,
    'Abs_Correlation': [correlations[f] for f in sorted_feature_names]
})

order_path = os.path.join(OUTPUT_DIR, "feature_order_by_correlation.xlsx")
order_df.to_excel(order_path, index=False)

print(f"\n 已保存重排后的训练集至: {train_out_path}")
print(f"已保存重排后的测试集至: {eval_out_path}")
print(f"特征排序名单已保存至: {order_path}")

# 预览
print("\n 训练集前 3 行预览（按相关性重排后）:")
for i in range(3):
    print(f"    {train_strings.iloc[i]} -> {train_df[TARGET_COL].iloc[i]}")

.....

不排序，直接按原始特征顺序将每行样本的特征值拼接成字符串，保存为 Excel。
.....

import pandas as pd
import os

# ===== 文件路径 =====
TRAIN_CSV = r"C:\Users\16532\Downloads\train (1).csv"
EVAL_CSV = r"C:\Users\16532\Downloads\eval (1).csv"
OUTPUT_DIR = r"E:\liangzi dat\统计"

```

```

os.makedirs(OUTPUT_DIR, exist_ok=True)

# 16 个特征列名 (原始顺序)
FEATURE_COLS = [
    'Palpitations', 'Sleep_Issues', 'Headaches', 'Irritability',
    'Concentration', 'Low_Mood', 'Health_Issues', 'Loneliness',
    'Peer_Comp', 'Prof_Issues', 'Work_Env', 'Relax_Struggle',
    'Home_Env', 'Acad_Conf', 'Subj_Conf', 'Act_Conflict'
]
TARGET_COL = 'Recent_Stress'

# ===== 读取数据 =====
train_df = pd.read_csv(TRAIN_CSV)
eval_df = pd.read_csv(EVAL_CSV)

# ===== 直接拼接字符串 (不排序) =====
def concat_features(df, feature_order):
    sub_df = df[feature_order].astype(str)
    return sub_df.apply(lambda row: ''.join(row), axis=1)

train_strings = concat_features(train_df, FEATURE_COLS)
eval_strings = concat_features(eval_df, FEATURE_COLS)

# 构建输出 DataFrame
train_out_df = pd.DataFrame({
    'Feature_String': train_strings,
    TARGET_COL: train_df[TARGET_COL]
})
eval_out_df = pd.DataFrame({
    'Feature_String': eval_strings,
    TARGET_COL: eval_df[TARGET_COL]
})

# ===== 保存文件 =====
train_out_path = os.path.join(OUTPUT_DIR, "train_feature_strings_original.xlsx")
eval_out_path = os.path.join(OUTPUT_DIR, "eval_feature_strings_original.xlsx")
train_out_df.to_excel(train_out_path, index=False)
eval_out_df.to_excel(eval_out_path, index=False)

print(f"已保存原始顺序拼接的训练集至: {train_out_path}")
print(f"已保存原始顺序拼接的测试集至: {eval_out_path}")

# 预览

```

```

print("\n 训练集前 3 行预览 (原始特征顺序) :")
for i in range(3):
    print(f"    {train_strings.iloc[i]} -> {train_df[TARGET_COL].iloc[i]}")

```

8.3 经典方法

使用排序特征字符串，字符级嵌入，加权 MSE 损失，阈值搜索

```

"""

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, accuracy_score, classification_report
import matplotlib.pyplot as plt
import os
import warnings
warnings.filterwarnings('ignore')

# ===== 配置 =====
TRAIN_PATH = "train_feature_strings_sorted.xlsx" # 请修改为实际路径
EVAL_PATH = "eval_feature_strings_sorted.xlsx" # 请修改为实际路径
OUTPUT_DIR = "BiLSTM_Results"
os.makedirs(OUTPUT_DIR, exist_ok=True)

SEQ_LEN = 16
VOCAB_SIZE = 6 # 字符 '1'~'5' 索引 1~5, 0 为 padding (实际不用)
EMBED_DIM = 32
HIDDEN_DIM = 64
NUM_LAYERS = 2
DROPOUT = 0.3
BATCH_SIZE = 32
EPOCHS = 200
LR = 0.001
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"使用设备: {DEVICE}")

# ===== 数据集类 =====

```

```

class StressDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.long)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

# ===== 模型定义 =====
class BiLSTMAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, dropout, seq_len):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                             batch_first=True, bidirectional=True, dropout=dropout)
        self.attention = nn.Linear(hidden_dim * 2, 1)
        self.dropout = nn.Dropout(dropout)
        self.fc1 = nn.Linear(hidden_dim * 2, 32)
        self.fc2 = nn.Linear(32, 16)
        self.out = nn.Linear(16, 1)

    def forward(self, x):
        x = self.embedding(x)  # (B, L, E)
        lstm_out, _ = self.lstm(x)  # (B, L, 2*H)

        # 注意力机制
        attn_weights = torch.softmax(self.attention(lstm_out).squeeze(-1), dim=1)  # (B, L)
        context = torch.sum(lstm_out * attn_weights.unsqueeze(-1), dim=1)  # (B, 2*H)

        context = self.dropout(context)
        x = torch.relu(self.fc1(context))
        x = self.dropout(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout(x)
        return self.out(x).squeeze(-1)

# ===== 加权 MSE 损失 =====
def weighted_mse_loss(pred, target):
    diff = pred - target
    w1 = (target == 1.0).float() * 2.5
    w5 = (target == 5.0).float() * 2.5
    w_other = ((target != 1.0) & (target != 5.0)).float() * 1.0
    weights = w1 + w5 + w_other
    return (weights * diff ** 2).mean()

```



```

# ===== 数据加载 =====
def load_data(train_path, eval_path):
    train_df = pd.read_excel(train_path)
    eval_df = pd.read_excel(eval_path)

    def string_to_int_array(s):
        return np.array([int(c) for c in s], dtype=np.int64)

    X_train = np.array([string_to_int_array(s) for s in train_df['Feature_String']])
    y_train = train_df['Recent_Stress'].values.astype(np.float32)
    X_test = np.array([string_to_int_array(s) for s in eval_df['Feature_String']])
    y_test = eval_df['Recent_Stress'].values.astype(np.float32)

    return X_train, y_train, X_test, y_test

print("加载数据...")
X_train_full, y_train_full, X_test, y_test = load_data(TRAIN_PATH, EVAL_PATH)

# 划分训练/验证集
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.2, random_state=42, stratify=y_train_full
)

print(f"训练集: {X_train.shape}, 验证集: {X_val.shape}, 测试集: {X_test.shape}")

train_dataset = StressDataset(X_train, y_train)
val_dataset = StressDataset(X_val, y_val)
test_dataset = StressDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# ===== 初始化模型 =====
model = BiLSTMAttention(
    vocab_size=VOCAB_SIZE,
    embed_dim=EMBED_DIM,
    hidden_dim=HIDDEN_DIM,
    num_layers=NUM_LAYERS,
    dropout=DROPOUT,
    seq_len=SEQ_LEN
).to(DEVICE)

optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=15, min_lr=1e-5)

# ===== 训练 =====

```

```

print("开始训练...")
train_losses, val_losses = [], []
best_val_loss = float('inf')
best_model_state = None

for epoch in range(EPOCHS):
    model.train()
    train_loss = 0.0
    for x_batch, y_batch in train_loader:
        x_batch, y_batch = x_batch.to(DEVICE), y_batch.to(DEVICE)
        optimizer.zero_grad()
        preds = model(x_batch)
        loss = weighted_mse_loss(preds, y_batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        train_loss += loss.item() * x_batch.size(0)
    train_loss /= len(train_loader.dataset)
    train_losses.append(train_loss)

    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for x_batch, y_batch in val_loader:
            x_batch, y_batch = x_batch.to(DEVICE), y_batch.to(DEVICE)
            preds = model(x_batch)
            loss = weighted_mse_loss(preds, y_batch)
            val_loss += loss.item() * x_batch.size(0)
    val_loss /= len(val_loader.dataset)
    val_losses.append(val_loss)

    scheduler.step(val_loss)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_model_state = model.state_dict().copy()

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1:3d}/{EPOCHS} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | LR: {optimizer.param_groups[0]['lr']:.6f}")

# 加载最佳模型
model.load_state_dict(best_model_state)
model.eval()

# ===== 阈值搜索 =====
print("\n 在验证集上搜索最优阈值...")

```

```

val_preds = []
with torch.no_grad():
    for x_batch, _ in val_loader:
        x_batch = x_batch.to(DEVICE)
        pred = model(x_batch).cpu().numpy()
        val_preds.extend(pred)
val_preds = np.array(val_preds)

def find_best_thresholds(preds, y_true):
    best_mae = float('inf')
    best_thresh = None
    cand = np.arange(1.5, 5.0, 0.1)
    for t1 in cand:
        for t2 in cand[cand > t1]:
            for t3 in cand[cand > t2]:
                for t4 in cand[cand > t3]:
                    y_pred = np.digitize(preds, bins=[t1, t2, t3, t4]) + 1
                    mae = mean_absolute_error(y_true, y_pred)
                    if mae < best_mae:
                        best_mae = mae
                        best_thresh = (t1, t2, t3, t4)
    return best_thresh, best_mae

thresholds, val_mae = find_best_thresholds(val_preds, y_val)
print(f"最佳阈值: {thresholds}, 验证 MAE: {val_mae:.4f}")

# ===== 测试集评估 =====
test_preds = []
with torch.no_grad():
    for x_batch, _ in test_loader:
        x_batch = x_batch.to(DEVICE)
        pred = model(x_batch).cpu().numpy()
        test_preds.extend(pred)
test_preds = np.array(test_preds)

y_pred_test = np.digitize(test_preds, bins=thresholds) + 1
y_pred_test = np.clip(y_pred_test, 1, 5)

test_mae = mean_absolute_error(y_test, y_pred_test)
test_acc = accuracy_score(y_test, y_pred_test)
score = 30 * np.exp(-0.1 * test_mae)

print("\n" + "="*60)
print("BiLSTM + Attention 测试集评估")
print("="*60)
print(f"MAE: {test_mae:.4f}, Accuracy: {test_acc:.4f}, Score: {score:.2f}")
print(classification_report(y_test, y_pred_test, digits=4))

```

```
# 保存预测
eval_df = pd.read_excel(EVAL_PATH)
eval_df['Predicted_Stress'] = y_pred_test
eval_df[['Recent_Stress', 'Predicted_Stress']].to_excel(
    os.path.join(OUTPUT_DIR, "bilstm_predictions.xlsx"), index=False
)

# 绘制损失曲线
plt.figure(figsize=(10, 5))
plt.plot(train_losses, label='Train')
plt.plot(val_losses, label='Validation')
plt.legend()
plt.grid()
plt.savefig(os.path.join(OUTPUT_DIR, "loss_curve.png"))
plt.show()

print(f"\n 所有结果已保存至: {OUTPUT_DIR}")
```

1.